

RANKSTORE

A 400-Byte Persistent Rank Summary for Deferred Tensor Analysis

Observe every sample now; choose quantiles, groups, diagnostics, and quantization later

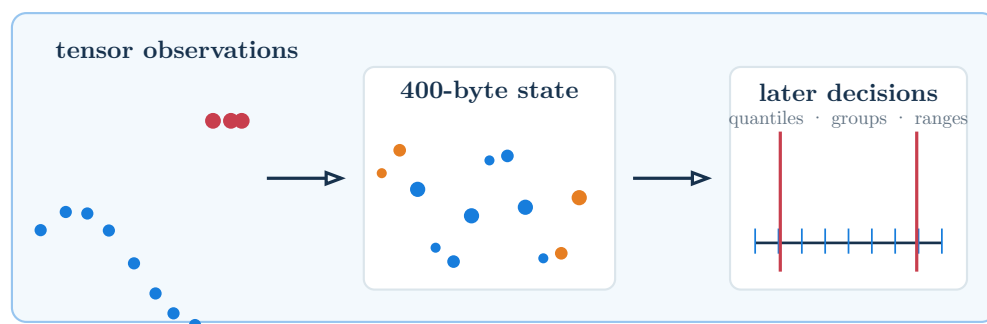
tensor histories

rank transport

exact ties

cold decisions

monatq research note · version 0.3



The proposal Replace the current per-position t-digest with one deterministic weighted approximation of the empirical tensor history. Sixty-four value knots, 16-bit probability masses, one tie mask, and exact extrema occupy 400 bytes of summary state per position. The live collector also needs an input buffer and bounded flush workspace. Collection commits to no percentile, grouping, visualization, or quantizer; those decisions remain cold and revisable.

An important wording distinction RANKSTORE **ingests every observation**, but it does not retain every raw value. It preserves an approximate empirical distribution from which later rank-based decisions can be made. Raw samples cannot be reconstructed or replayed.

Contents

1	Executive summary	2
2	Observe now, decide later	3
2.1	Why fixed summaries are often too early	3
2.2	What cannot be deferred	4
3	A compact empirical distribution	4
3.1	Positive weighted support	4
3.2	The 400-byte summary-state layout	4
4	Streaming compression	5
4.1	Buffer first, compress rarely	5
4.2	Tail-companded rank cells	5
4.3	Mixed cells use weighted means	5
4.4	Prefix-rounded probability	6
5	Cold reconstruction and regrouping	6
5.1	A monotone quantile curve	6
5.2	Choose robust ranges later	6
5.3	Merge arbitrary positions later	7
6	Example consumer: post-training quantization	7
6.1	Evaluate an affine quantizer	7
6.2	Search candidates on the stored measure	7
7	Accuracy measurement	8
7.1	Ties require rank intervals	8
7.2	Feasibility protocol	8
7.3	General-distribution results	8
7.4	Discrete observations	9
7.5	Important caveats	9
8	A deliberately small systems design	9
8.1	Rust representation	9
8.2	One compressor everywhere	10
8.3	Minimal configuration	10
9	Validation plan	10
9.1	Recorder-level gates	10
9.2	Consumer-level studies	10
10	Risks and limitations	11
11	Reproducibility	11
12	Conclusion	11
	References	12

1 Executive summary

TensorDigest receives a sequence of full tensor samples and tracks one distribution at every flat tensor position. The expensive part is not an individual value; it is the product of positions, samples, and persistent bytes. At collection time, callers often do not yet know which summaries they will need. They may later ask for robust ranges, percentiles, per-channel groups, visualizations, outlier diagnostics, or post-training quantization (PTQ) thresholds.

RANKSTORE separates **observation** from **decision**. During collection it maintains a compact positive approximation of each position’s empirical distribution. During a cold later stage it can answer quantiles, combine arbitrary positions into groups, evaluate range policies, or serve as input to a quantizer. Query and aggregation latency are explicitly secondary; update throughput and memory dominate.



Figure 1: The intended scope. Every tensor observation updates the same compact state. Later consumers can choose their own grouping and decision policy without replaying the tensor stream.

The proposed summary state uses 400 bytes per tensor position—roughly one twelfth of the repository t-digest’s 4,900-byte summary state at the default compression. It is 32 bytes, or 8.7%, larger than Quantile Spine’s 368-byte summary state. It contains no dynamic histogram, separately sized atom table, per-position heap allocation, or query-time cache. Equal values share the ordinary knot budget, and exact finite extrema remain available even though interior observations are compressed.

Memory scope The 400-byte figure is summary state, not complete live memory. With `f32` inputs, the proposed 256-row buffer adds 1,024 bytes per position. The previous comparison of 400 bytes against the t-digest’s complete 5.7 KiB allocation mixed those scopes and overstated the reduction.

Property	t-digest	Quantile Spine	RANKSTORE
Summary state/position	4,900 B	368 B	400 B
<code>f32</code> input buffer/position	800 B	1,024 B	1,024 B
Full sort buffer/position	none	1,024 B	none proposed
Tensor-scaled live total	≈ 5,700 B	2,416 B	1,424 B
State primitive	weighted centroids	rank anchors	weighted rank knots
Repeated values	interpolated	anchor dependent	retained tie interval
Exact min/max	implementation dependent	no	yes, by construction
Update	buffer + centroid merge	buffer + anchor blend	buffer + fixed-array rebin
Cold group merge	centroid merge	not available	union + same rebin

The table assumes default configurations, `f32`, a 64-bit target, and large tensors over which object headers and worker-local scratch amortize. The implemented t-digest and Quantile Spine live totals are verified from allocator-instrumented `backend_accuracy` runs rather than field-size estimates; the unimplemented RANKSTORE total remains a design projection. RANKSTORE’s 1,424-byte live estimate requires gathering and sorting into per-worker scratch rather than materializing a second tensor-sized buffer. Adding such a buffer would raise it to 2,448 bytes, slightly above the current Quantile Spine total. Conversely, Quantile Spine could adopt the same worker-local strategy and reach about 1,392 bytes. The live-memory difference between those two designs is therefore an implementation choice, not an inherent RANKSTORE advantage.

Evidence status Held-out Python experiments show better aggregate rank accuracy than the repository t-digest on smooth, atomic, quantized, and bimodal workloads. The 16-bit state transition and `f32` representatives are simulated. Rust throughput, exact live allocation, million-row drift, and downstream application quality remain unmeasured.

2 Observe now, decide later

2.1 Why fixed summaries are often too early

A minimum and maximum preserve endpoints but reveal nothing about how probability is distributed between them. A fixed histogram commits to bin edges before the eventual query is known. A single percentile commits to one tail policy. A preselected channel grouping prevents later comparison with per-tensor or hardware-aligned alternatives.

RANKSTORE instead records a reusable approximation of rank mass. The same collected state can support several later consumers:

- median, percentile bands, and robust ranges;
- per-position, per-channel, per-token, or whole-tensor aggregation;
- distribution plots and saturation diagnostics;
- anomaly or alert thresholds chosen after collection;
- PTQ clipping and scale selection;
- comparisons between alternative policies without replaying the model or data source.

The architecture is therefore closer to a compact tensor history than to a quantizer. PTQ is an important consumer, not the identity of the backend.

2.2 What cannot be deferred

No fixed-memory summary retains raw sample identity, temporal order, correlations between positions, or exact arbitrary quantiles. If a later analysis needs sample-level replay, covariance, or causality, it must store different information. Comparison-based quantile sketches necessarily trade memory for error [1]. RANKSTORE targets representative tensor distributions rather than a universal worst-case guarantee.

3 A compact empirical distribution

3.1 Positive weighted support

An observed stream defines an empirical distribution made of point masses. RANKSTORE replaces that large measure with at most 64 positive weighted locations. In one dimension, this can be understood as moving nearby rank mass onto representative support points—a small transport coreset [2].

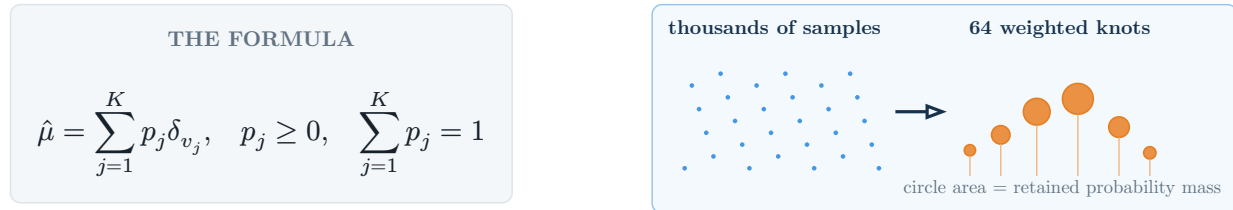


Figure 2: **The compressed history.** Thousands of observations become a short weighted support. Large circles carry more probability; positivity and normalization are explicit.

Every input affects the state, but most inputs cease to be individually identifiable after compression. Any number of distinct values may enter. At most 64 locations remain exact; excess support is approximated by the same rule used for continuous data. There is no separate “number of atoms supported” configuration.

3.2 The 400-byte summary-state layout

All positions receive one observation at the same logical tensor update. With NaNs rejected for the whole update, the accepted sample count can live once at `TensorDigest` level. The per-position summary state spends its final eight bytes on exact extrema instead. Input buffering and flush workspace are separate from this layout.

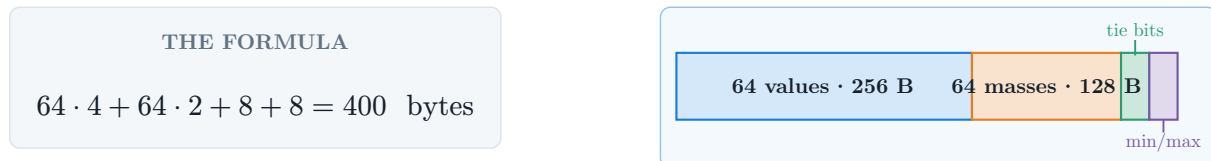


Figure 3: **Summary-state accounting.** The four terms are f32 locations, u16 masses, a 64-bit tie mask, and two f32 extrema. Zero mass identifies an unused slot.

Field	Bytes	Meaning
values[64]: f32	256	mixed-group means or retained exact values
masses[64]: u16	128	probability quanta; active masses sum to 65,535
pure_mask: u64	8	one bit per location that still represents one exact value
min, max: f32	8	exact finite endpoints for later range decisions
Summary-state total	400	no pointers and no per-position allocation

Per-position missingness would break the shared-count assumption and must pay an explicit memory or semantic cost. The initial backend should reject a whole tensor update containing NaN rather than silently desynchronize position counts.

4 Streaming compression

4.1 Buffer first, compress rarely

Updates append full row-major tensor samples to one 256-row input buffer. A flush gathers one position into worker-local scratch, then handles positions independently and in parallel:

1. Sort and run-length encode the new values.
2. Expand old normalized masses relative to the shared historical count.
3. Merge at most 64 old locations with the batch runs.
4. Place 64 target cells in rank space and snap cuts to indivisible masses.
5. Store a weighted mean for every mixed cell and a purity bit for every one-value cell.
6. Prefix-round cumulative masses back to 16 bits and update exact extrema.

The hot path uses bounded arrays over roughly 320 sorted entries. This scratch scales with the worker count, not the tensor width. The proposed implementation must not retain a `numel × batch_rows` sorted copy: that extra tensor-sized buffer would add another 1,024 bytes per f32 position. There is no per-position hash table, dynamic allocation, iterative optimization, or query reconstruction.

4.2 Tail-companded rank cells

Uniform rank cells spend the same resolution at the median and at the extremes. Many later decisions—robust ranges, alarms, saturation estimates, and quantization—care about tails. RANKSTORE therefore places fixed slots uniformly in an abstract coordinate that bends toward probability zero and one.

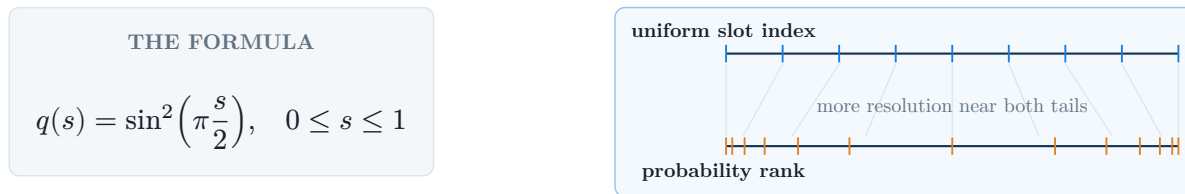


Figure 4: **Tail-aware rank placement.** Equal slot steps on the upper line become denser at both ends of probability space. This is the same broad tail-allocation principle that benefits t-digest [3].

A large exact tie may contain several desired cuts. Those cuts snap to the tie boundaries; unused slots then recursively split the widest remaining intervals in transformed rank space. A 50% zero mass consumes one support location rather than half the state.

4.3 Mixed cells use weighted means

A cell containing several locations stores their weighted mean. It is the constant representative minimizing squared value reconstruction error, and masses plus first moments combine cleanly before the next lossy repartition.

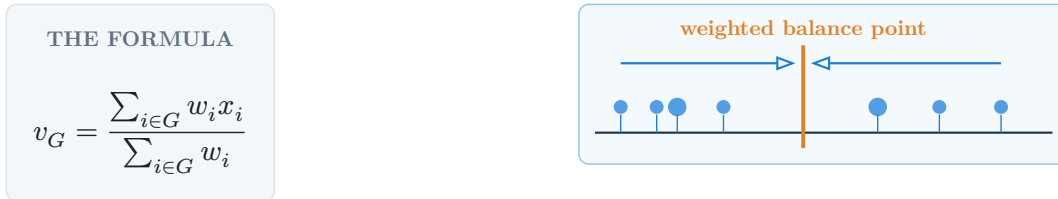


Figure 5: **One representative per mixed cell.** The orange line is the weighted balance point. In the feasibility prototype, medians produced materially larger smooth-distribution error under cold interpolation.

A cell containing one exact location retains that location and sets its purity bit. A mixed cell remains mixed even if its rounded mean happens to equal an input value, preventing accidental invention of a tie.

4.4 Prefix-rounded probability

Sixteen-bit probability masses allow 64 locations at the same memory where 32-bit counts allow 48. The exact total sample count remains outside the cell. Cumulative prefixes are rounded first, then differenced, so every represented CDF boundary is directly controlled.

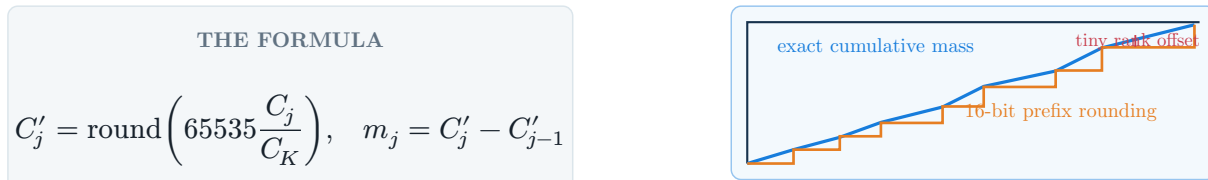


Figure 6: **Sixteen-bit mass encoding.** Each orange step follows the blue cumulative mass. Prefix rounding avoids independent mass errors all drifting in one direction.

The current boundary discrepancy is below one half probability quantum. Repeated compression and requantization can still accumulate approximation; long streams and repeated distribution shifts remain mandatory tests.

5 Cold reconstruction and regrouping

5.1 A monotone quantile curve

Each mixed knot is anchored at the center rank of its encoded mass. Neighboring mixed anchors are joined linearly. A pure knot contributes a horizontal interval across its mass. Sorting values and ranks makes the resulting quantile curve monotone.

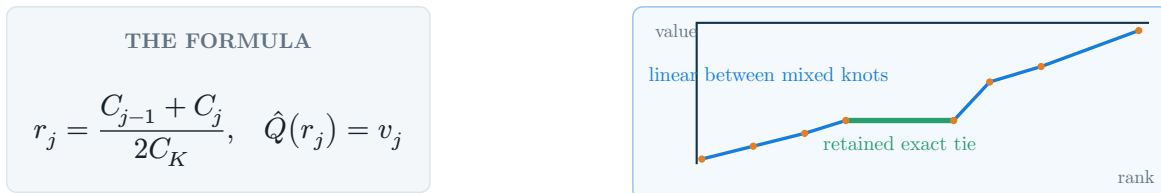


Figure 7: **Decode only when needed.** Blue segments interpolate unresolved continuous mass. The green plateau represents an exact retained tie across a whole rank interval.

Endpoint requests bypass this interpolation. Probability zero returns exact min; probability one returns exact max. This avoids the severe heavy-tail underestimation that would result from treating the outer mixed-group means as extrema.

5.2 Choose robust ranges later

A caller can defer its tail policy until after collection. One consumer may want a central visualization band; another may choose an anomaly threshold; another may estimate a clipping interval. All can use different tail probabilities from the same state.



Figure 8: **A deferred central range.** The shaded total tail mass is selected later. Collection does not hardcode τ , symmetry, or the eventual use of the range.

5.3 Merge arbitrary positions later

The stored object is a positive weighted measure. To form a channel, token group, spatial region, or whole tensor, scale cell masses by their observation counts and optional group weights, union all support, sort once, and run the same compressor.



Figure 9: **Deferred grouping.** Fine-grained histories become one group distribution without replaying the original tensor samples. Grouping policy is a cold decision.

Aggregation latency is intentionally not a performance requirement. Cold merges may allocate temporary vectors and globally sort all input support.

6 Example consumer: post-training quantization

PTQ illustrates why deferred distribution storage is useful. A calibration run can collect activation histories before choosing bit width, symmetric versus asymmetric ranges, per-tensor versus per-channel grouping, or clipping policy. The same RANKSTORE state can compare those choices later. PTQ methods commonly use a small representative set to establish activation ranges [4], [5].

6.1 Evaluate an affine quantizer

A bounded real interval determines the step spacing of a finite integer codebook [6], [7]. Wider endpoints preserve outliers but make every interior step coarser.

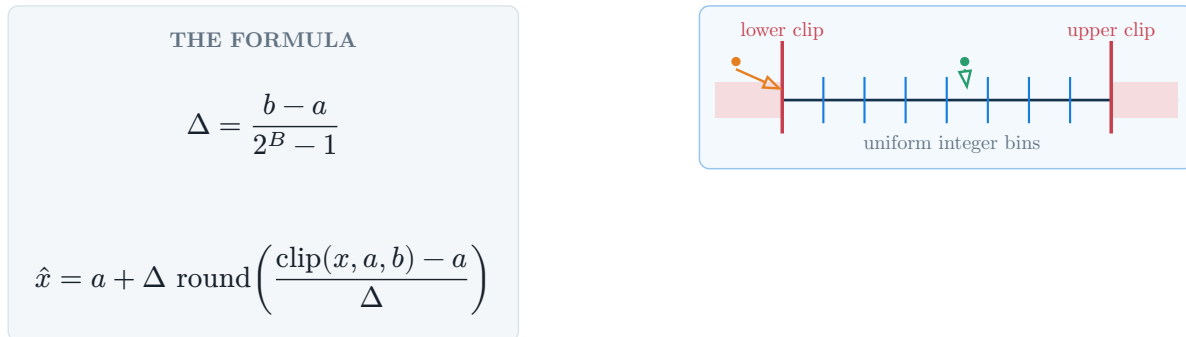


Figure 10: **One possible deferred quantizer.** The red endpoints and bit width are not chosen while observations are collected. They are candidate decisions evaluated afterward.

6.2 Search candidates on the stored measure

Instead of selecting one percentile in advance, a cold quantizer can evaluate candidate intervals directly on the 64 weighted locations. This approximates activation reconstruction error while making search cost independent of the original sample count.

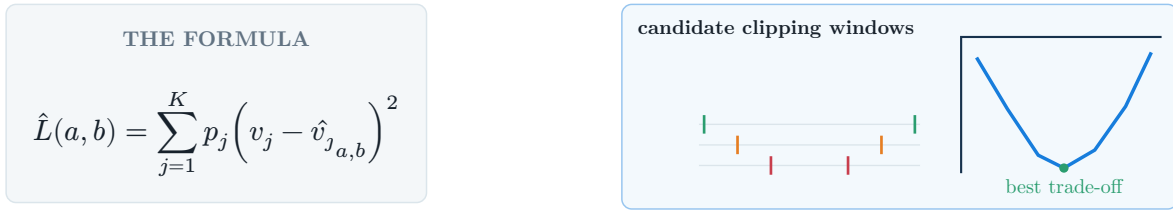


Figure 11: **Quantization is a consumer, not the state.** Narrow, medium, and wide candidate windows trade clipping against rounding. Other consumers can ignore this objective entirely.

This marginal objective does not model Hessians, cross-channel correlations, or final task loss. More sophisticated quantization methods can consume the same distributional evidence alongside their own model-specific information.

7 Accuracy measurement

7.1 Ties require rank intervals

A requested quantile inside an empirical jump is correctly answered by the repeated value. Measuring only one side of the CDF would report an error even though the returned value is a valid generalized quantile [8].

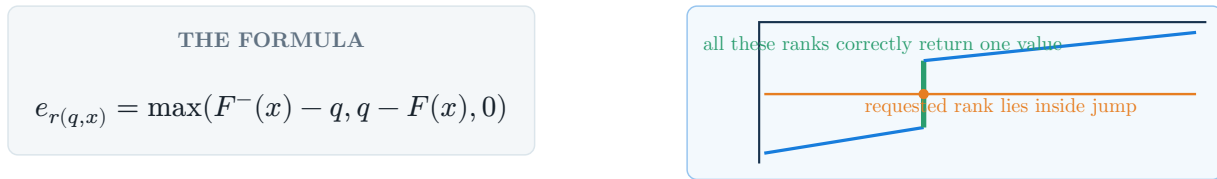


Figure 12: **Tie-aware rank error.** The orange requested rank lies inside the green CDF jump, so returning the repeated value has zero error.

Application-specific consumers must report their own outcomes as well. Rank error evaluates the recorder; it does not replace downstream quality, reconstruction error, alert precision, or visualization usefulness.

7.2 Feasibility protocol

The checked-in experiment uses two held-out seeds, 32,768 f32 observations per workload, 256-row flushes, and 235 query probabilities from one ten-thousandth through its upper complement. Parameter exploration used a separate seed. The Python prototype simulates 16-bit prefix masses and persistent f32 representatives. Repository **TDigest** and **QuantileSpine** run through a temporary Rust crate on identical ordered values.

Workloads include uniform, normal, two lognormal scales, Student-t, shuffled and blocked mixtures, one and five added atoms, 50% zero, constants, 8–256-level quantization, and identical uniform multisets in random, ascending, and descending order. Ordered cases are diagnostics rather than proposed gates.

7.3 General-distribution results

Workload	RANKSTORE mean	t-digest mean	RANKSTORE max	t-digest max
uniform	0.000267	0.001414	0.001150	0.005179
normal	0.000271	0.001018	0.001069	0.005746
lognormal, $\sigma=1$	0.000346	0.002420	0.001304	0.008954
lognormal, $\sigma=2$	0.000584	0.004453	0.001637	0.011169
Student-t, $df=2$	0.000392	0.000861	0.001288	0.004347
bimodal, shuffled	0.000927	0.009599	0.013983	0.066852
bimodal, blocked	0.001288	0.010447	0.016979	0.066882

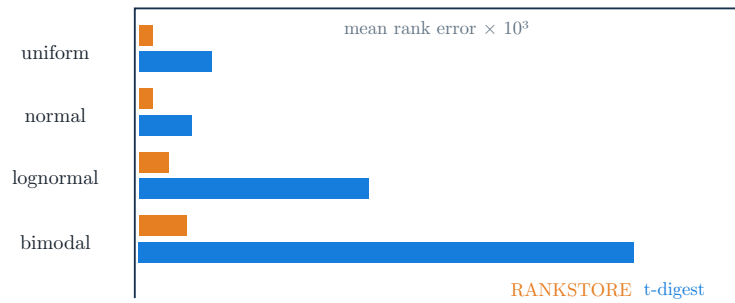


Figure 13: Mean valid-rank error on selected held-out workloads. Bars use thousandths of rank. Lower is better; separated modes remain harder than smooth distributions.

Across all 36 workload/seed pairs, RANKSTORE’s mean-of-means was 0.000330 and its worst observed maximum was 0.016979. The repository t-digest values were 0.006928 and 0.246068. RANKSTORE had lower mean and maximum error on every nonconstant pair; both methods tied on constants.

7.4 Discrete observations

Workload	Mean error	Worst error	Interpretation
five added atoms	0.000117	0.001653	ties share the ordinary knot budget
8 levels	0	0	all levels retained
16 levels	0	0	all levels retained
32 levels	0	0	all levels retained
64 levels	0	0	all levels retained in this test
256 levels	0.000831	0.002344	excess levels interpolated

Zero means zero at the 235 tested ranks, not equality at every possible probability. The result demonstrates that repeated values need no separately hardcoded capacity.

7.5 Important caveats

- **Extreme tails:** t-digest was often two to three times more accurate at individual smooth tail queries, although RANKSTORE’s absolute tail errors were small. The worst observed tail error was 0.001719.
- **Separated modes:** linear interpolation across a large gap remains the largest ordinary-distribution error.
- **Extrema:** the experiment’s interior state did not include endpoint requests. Proposed min/max sidecars make endpoints exact by construction without changing reported interior estimates.
- **No speed evidence:** Python runtime is not a proxy for Rust tensor throughput.
- **No consumer evidence:** no downstream quantizer, alert, or visualization study has yet compared decisions from this state.

8 A deliberately small systems design

8.1 Rust representation

```
#[repr(C)]
struct RankStore {
    values: [f32; 64],
    masses: [u16; 64],
    pure_mask: u64,
    min: f32,
    max: f32,
}
```

The accepted tensor sample count and exceptional-value policy belong to `TensorDigest`, shared across all positions. Temporary flush arithmetic uses wider accumulators; persistent representatives round to `f32` only once per flush. With the default 256 `f32` rows, live tensor-scaled storage is therefore $400 + 1,024 = 1,424$ bytes per position, not 400 bytes. Per-worker scratch and small object headers remain additional but do not scale with the number of positions.

8.2 One compressor everywhere

The same deterministic reducer should implement buffered flush and every cold merge. A single representation before and after aggregation avoids separate “streaming,” “query,” and “merge” models. Cold consumers may build caches, dense plots, or candidate-search workspaces, but none become persistent per-position state.

8.3 Minimal configuration

Parameter	Default	Reason
<code>knots</code>	64	fills the 400-byte packed layout
<code>mass_quanta</code>	65535	full positive <code>u16</code> probability range
<code>batch_rows</code>	256	matches the experiment; costs 1,024 input-buffer bytes per <code>f32</code> position
<code>rank_scale</code>	<code>arcsine</code>	balanced measured accuracy with tail emphasis
<code>NaN</code>	<code>reject update</code>	preserves one shared count across positions
<code>infinities</code>	<code>explicit endpoints</code>	avoid contaminating finite means

Do not expose the mass encoding or scale function publicly until long-stream and cross-platform tests stabilize the state transition.

9 Validation plan

9.1 Recorder-level gates

Gate	Requirement
Typical accuracy	lower aggregate mean and maximum rank error than <code>t-digest</code> on the preregistered representative suite
Memory	400 bytes of summary state and at most 1,424 tensor-scaled live bytes per <code>f32</code> position at 256 rows; no tensor-sized sorted copy
Hot throughput	update and flush throughput at least as high as the current <code>t-digest</code> at target tensor widths
Numerics	monotone queries, finite mixed means, exact finite extrema, and no mass underflow
Long streams	no unacceptable drift through at least one million updates and repeated distribution shifts
Cold merge	grouped summaries agree with direct-stream summaries within a declared rank tolerance

Ascending streams, blocked modes, and crafted float patterns remain non-gating diagnostics. They still need a loose catastrophic ceiling: an unlikely stream is not permission for NaNs or nonmonotone answers.

9.2 Consumer-level studies

Recorder accuracy is necessary but not sufficient. Validate at least one real deferred consumer from each intended class:

- a visualization or percentile-band report;
- a grouped channel/tensor summary;

- a PTQ calibration comparison against t-digest-selected ranges;
- a thresholding or saturation diagnostic if those APIs are planned.

The consumer study should use the same collected state to compare several policies. That is the central benefit of deferral.

10 Risks and limitations

- **Approximation, not archival:** individual observations and temporal order are irrecoverable.
- **Representative data:** no summary can infer a deployment regime that was never observed.
- **Marginal state:** cross-position covariance and sample identity are absent.
- **Requantization drift:** 16-bit masses buy 64 support locations but repeatedly approximate old probability. A 48-location u32 fallback remains useful.
- **Shared count:** rejecting an entire update because one position is NaN may not suit every caller. Per-position missingness requires a revised layout.
- **Tail parity:** t-digest remains stronger at many individual extreme-tail queries.
- **Working memory:** the 400-byte title names summary state only. The default input buffer raises proposed live tensor-scaled storage to 1,424 bytes per f32 position, and a full sorted copy would raise it to 2,448 bytes.
- **Fixed memory:** no 400-byte summary state offers tiny distribution-free error for every stream.

The design principle Collection should preserve a useful approximation of the observed distribution while making as few downstream choices as possible. RANKSTORE spends its state budget on rank mass, ties, and extrema; everything else is deferred.

11 Reproducibility

The executable experiment and held-out results are:

- docs/experiments/transport_coreset_feasibility.py
- docs/experiments/results/transport-coreset-holdout.csv

The script deterministically generates all workloads, simulates both 48-location u32 and 64-location u16 streaming layouts, creates a temporary Rust crate, and queries repository `TDigest` and `QuantileSpine`. It requires Python, NumPy, and Cargo. The offline row is a representation upper bound, not a streaming backend.

The next artifact should record real tensor shapes, sample counts, throughput, selected cold policies, and consumer outputs in a machine-readable table.

12 Conclusion

RANKSTORE is a compact tensor-history backend, not a quantizer. It ingests every observation into a 400-byte positive summary state, preserves retained ties and exact extrema, and allows quantiles, grouping, visualization, clipping, and quantization policies to be chosen later. In the current feasibility suite it improves broad rank accuracy and discrete behavior over the repository t-digest. Apples-to-apples, its summary state is about one twelfth of the default t-digest state; its proposed one-buffer live tensor-scaled allocation is about one quarter of the current t-digest allocation. Its summary state is 32 bytes larger than Quantile Spine's, so any live-memory advantage over that backend depends on avoiding Quantile Spine's tensor-sized sort buffer.

The remaining questions are operational: Rust throughput and exact live allocation, long-stream mass drift, cold-merge fidelity, and whether real deferred consumers make decisions as good as or better than those based on t-digest. Those tests should decide the backend before its public API is stabilized.

References

- [1] Z. Karnin, K. Lang, and E. Liberty, “Optimal Quantile Approximation in Streams,” in *2016 IEEE 57th Annual Symposium on Foundations of Computer Science*, 2016, pp. 71–78. doi: [10.1109/FOCS.2016.17](https://doi.org/10.1109/FOCS.2016.17).
- [2] G. Peyré and M. Cuturi, *Computational Optimal Transport*, vol. 11, no. 5–6. Now Publishers, 2019. doi: [10.1561/22000000073](https://doi.org/10.1561/22000000073).
- [3] T. Dunning and O. Ertl, “Computing Extremely Accurate Quantiles Using t-Digests,” *arXiv preprint arXiv:1902.04023*, 2019, doi: [10.48550/arXiv.1902.04023](https://doi.org/10.48550/arXiv.1902.04023).
- [4] I. Hubara, Y. Nahshan, Y. Hanani, R. Banner, and D. Soudry, “Accurate Post Training Quantization With Small Calibration Sets,” in *Proceedings of the 38th International Conference on Machine Learning*, in Proceedings of Machine Learning Research, vol. 139. 2021, pp. 4466–4475. [Online]. Available: <https://proceedings.mlr.press/v139/hubara21a.html>
- [5] M. Nagel, M. Fournarakis, R. A. Amjad, Y. Bondarenko, M. van Baalen, and T. Blankevoort, “A White Paper on Neural Network Quantization,” *arXiv preprint arXiv:2106.08295*, 2021, doi: [10.48550/arXiv.2106.08295](https://doi.org/10.48550/arXiv.2106.08295).
- [6] R. M. Gray and D. L. Neuhoff, “Quantization,” *IEEE Transactions on Information Theory*, vol. 44, no. 6, pp. 2325–2383, 1998, doi: [10.1109/18.720541](https://doi.org/10.1109/18.720541).
- [7] B. Jacob *et al.*, “Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference,” in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 2704–2713. [Online]. Available: https://openaccess.thecvf.com/content_cvpr_2018/html/Jacob_Quantization_and_Training_CVPR_2018_paper.html
- [8] R. J. Hyndman and Y. Fan, “Sample Quantiles in Statistical Packages,” *The American Statistician*, vol. 50, no. 4, pp. 361–365, 1996, doi: [10.1080/00031305.1996.10473566](https://doi.org/10.1080/00031305.1996.10473566).